x86-64-v3 Architectures

Rubén Glez Python Vigo feb 2026

Introducing architecture variants: amd64v3 now available in Ubuntu 25.10

Project Discussion ■ Foundations blog

mwhudson Oct 2025

Ubuntu prides itself on being among the most compatible Linux distributions. Compatibility is often a conscious trade-off against bleeding-edge performance. In Ubuntu 25.10, we have added support for packages that target specific silicon variants, meaning you can have your cake and eat it too!

Back in 2023 I wrote an article 699 talking about the history of the `amd64` / `x86-64` architecture and described the "levels" `x86-64-v2`, `-v3`, and `-v4` (often referred to as `amd64v3`, `amd64v4`, etc.). Since then, we've been working on a means to better exploit modern processors without compromising support for older hardware.

To do this, we have added the concept of an "architecture variant." By making changes to `dpkg`, `apt` and Launchpad, we are able to build multiple versions of a package, each for a different level of the `x86-64` architecture, meaning we can have packages that specifically target `x86-64-v3`, for example.

As a result, we're very excited to share that in Ubuntu 25.10, some packages are available, on an opt-in basis, in their optimized form for the more modern `x86-64-v3` architecture level.

What's been done?

```
rgonzalez@rgonzalez-ThinkPad-T14 ~> cpuinfo
Python Version: 3.14.3.final.0 (64 bit)
Cpuinfo Version: 9.0.0
Vendor ID Raw: AuthenticAMD
Hardware Raw:
Brand Raw: AMD Ryzen 7 PRO 7840U w/ Radeon 780M Graphics
Hz Advertised Friendly: 5.1349 GHz
Hz Actual Friendly: 5.1349 GHz
Hz Advertised: (5134889000, 0)
Hz Actual: (5134889000, 0)
Arch: X86_64
Bits: 64
Count: 16
Arch String Raw: x86_64
L1 Data Cache Size: 262144
L1 Instruction Cache Size: 262144
L2 Cache Size: 8388608
L2 Cache Line Size: 1024
L2 Cache Associativity: 6
L3 Cache Size: 1048576
Stepping: 1
Model: 116
Family: 25
Processor Type:
Flags: 3dnowext, 3dnowprefetch, abm, adx, aes, amd_lbr_pmc_freeze, amd_lbr_v2, aperfmperf, apic, arat, avx, avx2, avx512_bf16, a
vx512_bitalg, avx512_vbmi2, avx512_vnni, avx512_vpopcntdq, avx512bitalg, avx512bw, avx512cd, avx512dq, avx512f, avx512ifma, avx5
12vbmi, avx512vbmi2, avx512vl, avx512vnni, avx512vpopcntdq, bmi1, bmi2, bpext, cat_l3, cdp_l3, clflush, clflushopt, clwb, clzero
, cmov, cmp_legacy, constant_tsc, cpb, cppc, cpuid, cpuid_fault, cqm, cqm_llc, cqm_mbm_local, cqm_mbm_total, cqm_occup_llc, cr8_
legacy, cx16, cx8, dbx, de, decodeassists, erms, extapic, extd_apicid, f16c, flush_l1d, flushbyasid, fma, fpu, fsgsbase, fsrm, f
xsr, fxsr_opt, gfni, ht, hw_pstate, ibpb, ibrs, ibrs_enhanced, ibs, invpcid, irperf, lahf_lm, lbrv, lm, mba, mca, mce, misaligns
se, mmx, mmxext, monitor, movbe, msr, mtrr, mwaitx, nonstop_tsc, nopl, npt, nrip_save, nx, ospke, osvw, osxsave, overflow_recov,
 pae, pat, pausefilter, pci_l2i, pclmulqdq, pdpe1gb, perfctr_core, perfctr_llc, perfctr_nb, perfmon_v2, pfthreshold, pge, pku, p
ni, popcnt, pqe, pqm, pse, pse36, rapl, rdpid, rdpru, rdrand, rdrnd, rdseed, rdt_a, rdtscp, rep_good, sep, sha, sha_ni, skinit,
smap, smca, smep, ssbd, sse, sse2, sse4_1, sse4_2, sse4a, ssse3, stibp, succor, svm, svm_lock, syscall, tce, topoext, tsc, tsc_s
cale, umip, user_shstk, v_spec_ctrl, vaes, vgif, vmcb_clean, vme, vmmcall, vnmi, vpclmulqdq, wbnoinvd, wdt, x2apic, x2avic, xget
bv1, xsave, xsavec, xsaveerptr, xsaveopt, xsaves, xtopology
rgonzalez@rgonzalez-ThinkPad-T14 ~>
```

```
power management: ts ttp tm hwpstate cpb eff_freq_ro [13] [14] [15]
rgonzalez@rgonzalez-ThinkPad-T14 ~> more /proc/cpuinfo
processor       : 0
vendor_id       : AuthenticAMD
cpu family      : 25
model           : 116
model name      : AMD Ryzen 7 PRO 7840U w/ Radeon 780M Graphics
stepping        : 1
microcode       : 0xa70410a
cpu MHz         : 3257.028
cache size      : 1024 KB
physical id     : 0
siblings        : 16
core id         : 0
cpu cores       : 8
apicid          : 0
initial apicid  : 0
fpu             : yes
fpu_exception   : yes
cpuid level     : 16
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall n
x mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good amd_lbr_v2 nopl xtopology nonstop_tsc cpuid extd_apicid aperfmperf rap
l pni pclmulqdq monitor ssse3 fma cx16 sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx f16c rdrand lahf_lm cmp_legacy extapic cr
8_legacy abm sse4a misalignsse 3dnowprefetch osvw ibs skinit wdt tce topoext perfctr_core perfctr_nb bpext perfctr_llc mwaitx cp
uid_fault cpb cat_l3 cdp_l3 hw_pstate ssbd mba perfmon_v2 ibrs ibpb stibp ibrs_enhanced vmmcall fsgsbase bmi1 avx2 smep bmi2 erm
s invpcid cqm rdt_a avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsave
c xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_total cqm_mbm_local user_shstk avx512_bf16 clzero irperf xsaveerptr rdpru wbnoinv
d cppc arat npt lbrv svm_lock nrip_save tsc_scale vmcb_clean flushbyasid decodeassists pausefilter pfthreshold vgif x2avic v_spe
c_ctrl vnmi avx512vbmi umip pku ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid overflo
w_recov succor smca fsrm flush_l1d amd_lbr_pmc_freeze
bugs            : sysret_ss_attrs spectre_v1 spectre_v2 spec_store_bypass srso spectre_v2_user tsa vmscape
bogomips        : 6587.63
TLB size        : 3584 4K pages
clflush size    : 64
cache_alignment : 64
address sizes   : 48 bits physical, 48 bits virtual
```

```
rgonzalez@rgonzalez-ThinkPad-T14 ~> /usr/lib/ld-linux-x86-64.so.2 --help
Usage: /usr/lib/ld-linux-x86-64.so.2 [OPTION]... EXECUTABLE-FILE [ARGS-FOR-PROGRAM...]
You have invoked 'ld.so', the program interpreter for dynamically-linked
ELF programs.  Usually, the program interpreter is invoked automatically
when a dynamically-linked executable is started.

You may invoke the program interpreter program directly from the command
line to load and run an ELF executable file; this is like executing that
file itself, but always uses the program interpreter you invoked,
instead of the program interpreter specified in the executable file you
run.  Invoking the program interpreter directly provides access to
additional diagnostics, and changing the dynamic linker behavior without
setting environment variables (which would be inherited by subprocesses).

  --list                list all dependencies and how they are resolved
  --verify              verify that given object really is a dynamically linked
                        object we can handle
  --inhibit-cache       Do not use /etc/ld.so.cache
  --library-path PATH   use given PATH instead of content of the environment
                        variable LD_LIBRARY_PATH
  --glibc-hwcaps-prepend LIST
                        search glibc-hwcaps subdirectories in LIST
  --glibc-hwcaps-mask LIST
                        only search built-in subdirectories if in LIST
  --inhibit-rpath LIST  ignore RUNPATH and RPATH information in object names
                        in LIST
  --audit LIST          use objects named in LIST as auditors
  --preload LIST        preload objects named in LIST
  --argv0 STRING        set argv[0] to STRING before running
  --list-tunables       list all tunables with minimum and maximum values
  --list-diagnostics    list diagnostics information
  --help                display this help and exit
  --version             output version information and exit

This program interpreter self-identifies as: /usr/lib/ld-linux-x86-64.so.2

Shared library search path:
  (libraries located via /etc/ld.so.cache)
  /usr/lib (system search path)

Subdirectories of glibc-hwcaps directories, in priority order:
  x86-64-v4 (supported, searched)
  x86-64-v3 (supported, searched)
  x86-64-v2 (supported, searched)
rgonzalez@rgonzalez-ThinkPad-T14 ~>
```

Level	Key Extensions & Features	Target Era / CPUs
x86-64-v2	SSE3, SSSE3, SSE4.1, SSE4.2, POPCNT, CX16 (CMPXCHG16B)	2008+ (Intel Nehalem, AMD Bulldozer)
x86-64-v3	AVX, AVX2, BMI1, BMI2, F16C, FMA, LZCNT, MOVBE	2013+ (Intel Haswell, AMD Excavator)
x86-64-v4	AVX-512 (F, BW, CD, DQ, VL)	2017+ (Intel Skylake-X/ICL, AMD Zen 4)

~/t/pyvigo26feb

```
rgonzalez@rgonzalez-ThinkPad-T14 ~/t/pyvigo26feb> cat Dockerfile
```

File: **Dockerfile**

```
# Use a last future LTS version of Ubuntu
FROM ubuntu:26.04

# Prevent interactive prompts during package installation
ENV DEBIAN_FRONTEND=noninteractive

# Install system dependencies
RUN apt-get update && apt-get install -y \
    python3 python3-pip pipx \
    ninja-build build-essential \
    git time \
    ripgrep emacs-nox

RUN pipx install meson pyperformance

#RUN git clone https://github.com/mesonbuild/meson.git


# cd meson && ./run_tests.py
# /usr/bin/time -v  /root/.local/bin/pyperformance
```

```
rgonzalez@rgonzalez-ThinkPad-T14 ~/t/pyvigo26feb>
```

~/t/pyvigo26feb

```
rgonzalez@rgonzalez-ThinkPad-T14 ~/t/pyvigo26feb>
rgonzalez@rgonzalez-ThinkPad-T14 ~/t/pyvigo26feb> difft Dockerfile Dockerfile_amd64v3
Dockerfile --- Text
 4 # Prevent interactive prompts during package installation      4 # Prevent interactive prompts during package installation
 5 ENV DEBIAN_FRONTEND=noninteractive                             5 ENV DEBIAN_FRONTEND=noninteractive
 6                                                                6
 .                                                                7 # Enable new architecture variants amd64v3
 .                                                                8 RUN echo 'APT::Architecture-Variants "amd64v3";' | tee /etc/ap
 .                                                                . t/apt.conf.d/99enable-amd64v3
 .                                                                9
 7 # Install system dependencies                                 10 # Install system dependencies
 8 RUN apt-get update && apt-get install -y \                    11 RUN apt update && apt upgrade -y && apt-get install -y \
 9     python3 python3-pip pipx \                                12     python3 python3-pip pipx \
10     ninja-build build-essential \                             13     ninja-build build-essential \
11     git time \                                                14     git time \

rgonzalez@rgonzalez-ThinkPad-T14 ~/t/pyvigo26feb>
```

pyperformance run -b "xml_etree,fastapi,django_template"

ubuntu:26.04

```
Performance version: 1.14.0
Python version: 3.13.12 (64-bit)
Report on Linux-6.18.9-arch1-2-x86_64-with-glibc2.42
Number of logical CPUs: 16
Start date: 2026-02-19 18:20:36.528063
End date: 2026-02-19 18:22:43.318220

### django_template ###
Mean +- std dev: 21.1 ms +- 0.4 ms

### fastapi_http ###
Mean +- std dev: 181 ms +- 9 ms

### xml_etree_parse ###
Mean +- std dev: 90.9 ms +- 3.5 ms

### xml_etree_iterparse ###
Mean +- std dev: 61.4 ms +- 3.3 ms

### xml_etree_generate ###
Mean +- std dev: 51.6 ms +- 1.9 ms

### xml_etree_process ###
Mean +- std dev: 36.3 ms +- 1.6 ms
```

ubuntu:26.04 con amd64v3

```
Performance version: 1.14.0
Python version: 3.13.12 (64-bit)
Report on Linux-6.18.9-arch1-2-x86_64-with-glibc2.42
Number of logical CPUs: 16
Start date: 2026-02-19 18:13:01.139293
End date: 2026-02-19 18:15:12.217879

### django_template ###
Mean +- std dev: 20.7 ms +- 0.4 ms

### fastapi_http ###
Mean +- std dev: 170 ms +- 13 ms

### xml_etree_parse ###
Mean +- std dev: 84.4 ms +- 2.6 ms

### xml_etree_iterparse ###
Mean +- std dev: 56.7 ms +- 2.3 ms

### xml_etree_generate ###
Mean +- std dev: 50.0 ms +- 1.2 ms

### xml_etree_process ###
Mean +- std dev: 34.9 ms +- 0.7 ms
```

cachyos/cachyos-v4

```
Performance version: 1.14.0
Python version: 3.14.3 (64-bit)
Report on Linux-6.18.9-arch1-2-x86_64-with-glibc2.43
Number of logical CPUs: 16
Start date: 2026-02-19 18:08:04.810911
End date: 2026-02-19 18:10:15.668499

### django_template ###
Mean +- std dev: 23.2 ms +- 0.5 ms

### fastapi_http ###
Mean +- std dev: 195 ms +- 20 ms

### xml_etree_parse ###
Mean +- std dev: 93.7 ms +- 8.9 ms

### xml_etree_iterparse ###
Mean +- std dev: 59.0 ms +- 5.2 ms

### xml_etree_generate ###
Mean +- std dev: 54.1 ms +- 1.1 ms

### xml_etree_process ###
Mean +- std dev: 38.7 ms +- 1.0 ms
```

Conclusiones

- Antes de optimizar, mide mucho y bien.
- Conoce tus herramientas
- La optimización prematura es la raíz de todos los males. (Donald Knuth)
- You can optimise for execution speed. You can optimise for space. But the most precious thing you should optimise for is understandability
- Link Time Optimization (LTO) y -O3 tienen mejores resultados
- https://www.phoronix.com/review/ubuntu-2510-amd64v3
- https://www.phoronix.com/news/Ubuntu-Server-25.10-amd64v3